

ViT 커스텀 데이터 Inference - 이미지 및 카메라

JD-edu

2025 All right reserved

로컬 디스크 이미지 추론

```
import torch

from PIL import Image

from transformers import ViTForImageClassification, ViTImageProcessor

import time

import os

MODEL_PATH = "./final_custom_vit_model"

IMAGE_FILE_PATH = "./cans.jpg"
```

모델 로드 및 초기화

```
try:
    if not os.path.isdir(MODEL_PATH):
        raise FileNotFoundError(f"모델 폴더를 찾을 수 없습니다: {MODEL_PATH}")

    processor = ViTImageProcessor.from_pretrained(MODEL_PATH)
    model = ViTForImageClassification.from_pretrained(MODEL_PATH)
    # GPU 사용 가능 여부 확인 후 디바이스 설정 (CPU 환경을 가정)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device)

    model.eval() # 추론 모드로 설정

    print(f" 모델 로드 완료. 현재 Device: {device}")
except FileNotFoundError as e:
    print(f"오류: {e}")
    print("모델 저장 폴더와 파일(pytorch_model.bin, config.json 등)의 위치를 확인해주세요.")
    exit()
except Exception as e:
    print(f"모델 로드 중 예기치 않은 오류 발생: {e}")
    exit()
```

이미지 전처리 및 예측 수행

```
try:

    start_time = time.time()

    image = Image.open(IMAGE_FILE_PATH).convert("RGB")

    inputs = processor(images=image, return_tensors="pt").to(device)

    # 5. 예측 (Prediction) 수행

    with torch.no_grad():

        outputs = model(**inputs)

    # 6. 결과 해석

    probabilities = torch.nn.functional.softmax(outputs.logits, dim=-1)

    predicted_prob = probabilities.max().item()

    predicted_class_idx = probabilities.argmax(-1).item()

    predicted_label = model.config.id2label[predicted_class_idx]

    end_time = time.time()

    print("\n--- 추론 결과 ---")

    print(f"소요 시간: {end_time - start_time:.4f} 초")

    print(f"예측된 클래스: **{predicted_label}**")

    print(f"예측 확률: {predicted_prob:.4f}")

except FileNotFoundError:

    print(f" 오류: 이미지 파일 '{IMAGE_FILE_PATH}'을(를) 찾을 수 없습니다. 경로를 다시 확인해주세요.")

except Exception as e:

    print(f" 추론 과정 중 오류 발생: {e}")
```



카메라 이미지 추론

```
import torch
from PIL import Image
from transformers import ViTForImageClassification, ViTImageProcessor
import cv2 # 웹캠 사용을 위한 OpenCV
import numpy as np # 이미지 변환을 위한 NumPy
import os
import time
MODEL_PATH = "./final_custom_vit_model" # 학습 후 모델이 저장된 폴더 경로
```

카메라 모델 로드

```
try:
    if not os.path.isdir(MODEL_PATH):
        raise FileNotFoundError(f"모델 폴더를 찾을 수 없습니다: {MODEL_PATH}")
    # 프로세서와 모델 로드
    processor = ViTImageProcessor.from_pretrained(MODEL_PATH)
    model = ViTForImageClassification.from_pretrained(MODEL_PATH)
    # CPU 사용을 기본으로 설정합니다. (GPU 사용 가능 시 'cuda'로 자동 설정됨)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device) model.eval() # 추론 모드로 설정
    print(f"모델 로드 완료. 현재 Device: {device}")
except Exception as e:
    print(f"오류: 모델 로드 중 오류 발생: {e}")
    print("모델 저장 폴더(Vision_transformer_cstom)와 파일이 올바른 위치에 있는지 확인하세요.")
    exit()
```



웹캠 초기화 및 라이브 스트림 시작

```
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print(" 오류: 웹캠을 열 수 없습니다. 카메라 연결을 확인하세요.")
    exit()
print("\n--- ViT 라이브 스트림 분류 시작 (ESC 또는 'q' 키를 누르면 종료) ---")
```

프레임 처리 및 예측

```
while True:
    start_time = time.time()
    # 프레임 읽기
    ret, frame = cap.read()
    if not ret:
        print("프레임을 읽을 수 없습니다.")
        break

    # 1) OpenCV BGR 형식을 ViT 입력에 필요한 RGB PIL Image로 변환
    frame_rgb_np = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    image_pil = Image.fromarray(frame_rgb_np)

    # 2) ViT 프로세서로 전처리 및 텐서 생성 # processor는 저장된 설정(Resize, Normalize)에 따라 이미지를 변환합니다.
    inputs = processor(images=image_pil, return_tensors="pt").to(device)

    # 3) 예측 (Prediction) 수행
    with torch.no_grad():
        outputs = model(**inputs)
```


결과 표시 화면 출력

4) 결과 해석: 로짓 -> 확률 -> 클래스 인덱스

```
probabilities = torch.nn.functional.softmax(outputs.logits, dim=-1)
```

```
predicted_class_idx = probabilities.argmax(-1).item()
```

```
predicted_prob = probabilities.max().item()
```

5) 클래스 이름으로 변환

```
predicted_label = model.config.id2label[predicted_class_idx]
```

```
end_time = time.time() fps = 1 / (end_time - start_time)
```

화면 출력

```
# --- 6. 결과 화면에 표시 (OpenCV) ---
# 예측 라벨 텍스트
text = f"Pred: {predicted_label} ({predicted_prob*100:.1f}%"
# FPS 텍스트
fps_text = f"FPS: {fps:.2f}"
# 텍스트 오버레이 (초록색)
cv2.putText(frame, text, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2, cv2.LINE_AA)
# FPS 오버레이 (노란색)
cv2.putText(frame, fps_text, (10, 60), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2, cv2.LINE_AA)
# 화면에 프레임 표시
cv2.imshow('ViT Custom Live Classification', frame)
# 7. 종료 조건 # 'q' 키 또는 ESC 키를 누르면 루프 종료
key = cv2.waitKey(1) & 0xFF
if key == ord('q') or key == 27:
    break
```